

Mesos Final Project Report

Team 11: Martino, Mazzucato, Mezzaroma, Palla

1. Implemented Features

The project successfully implements the digital version of the Mesos board game according to the provided requirements. The implemented features include:

- **Game Rules:** Complete Rules, supporting the full base game experience.
- **User Interface:** Both TUI and GUI are fully functional.
- **Networking:** Full support for distributed play over both RMI and TCP Socket protocols.
- **Advanced Features (FA):** implemented games leaderboard using an external database and the server supports concurrent game management, allowing multiple games to run simultaneously.

2. Division of Labor

The development workload was distributed equally among the team members. To ensure each member was fully aware of every part of the project, continuous communication has been key and every side-aspect of the project has been curated by at least two people simultaneously. The main core game logic has been defined by all the team in unison. The primary areas of focus were divided as follows:

- *Davide Mezzaroma:* game board definition and event management in Model module, Networking and Controller packages and TUI view.
- *Tommaso Palla:* building management in Model module, TUI view and Controller packages, Database Functionality.
- *Lorenzo Mazzucato:* Game and Player entities in Model package, GUI view, Client-side controller and model.
- *Paolo Martino:* deck and cards entities with parsing mechanism in Model package, Client controller, game scene in GUI View with related utils.

3. Implementation Choices & Design Patterns

The system was designed strictly adhering to object-oriented principles, ensuring high cohesion, low coupling, and scalability. Key design patterns used include:

- **MVC (Model-View-Controller):** Serves as the macro-architecture for the entire application, neatly decoupling the game state (Model) from the user interface (View) and the game logic/input handling (Controller). It was made use of a Thick Client architecture, where a Client Controller manages the communication between a client and the server, also containing some of the game logic. This allows to reduce the number of messages between Client and Server. A Server Controller manages all the instances of active games, and routes every request from a client to the respective Game Controller, which will handle the request, update the model status and notify the players (if necessary).
- **Strategy:** Implemented to handle the varying logic of game *Events* (e.g., Sustenance,

Hunt) and *Building* effects. This allows the game to calculate effects dynamically without heavy switch statements.

- **Visitor:** Used for operations that require traversing the player's tribe or game board, applying different logic depending on the specific type of card or token encountered.
- **Factory:** Extensively utilized for parsing game data (cards, buildings) and generating objects. It was also critical for parsing and generating Socket messages consistently.
- **Command:** Deployed within the TUI to encapsulate user console inputs into distinct executable objects, standardizing the execution of player actions.
- **DTO (Data Transfer Object):** Deployed to encapsulate game state data and player actions into lightweight, serializable packets. This ensures that all message exchanges traversing the TCP Socket layer remain entirely decoupled from the core domain logic.
- **DAO (Data Access Object):** Used to implement the Advanced Feature (Database Leaderboard), abstracting and encapsulating all access to the underlying database.

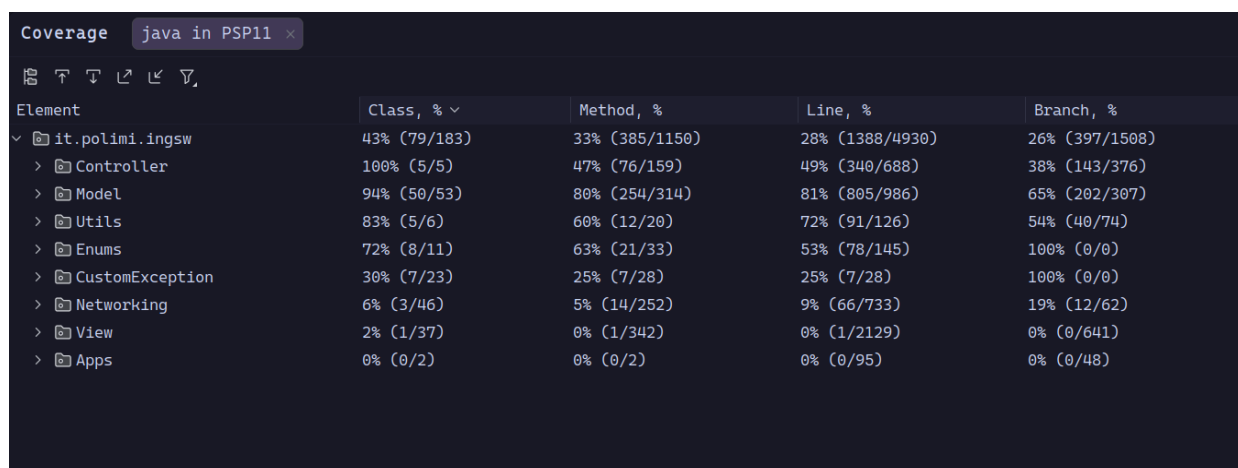
4. Communication Protocols

The distributed nature of the application is handled via two parallel communication protocols, transparently abstracted to the Controller layer.

- **RMI:** Used for synchronous, transparent communication. Stubs handle the remote references natively within the Java environment.
- **TCP Socket:** A custom asynchronous message-passing protocol. Messages are serialized and parsed back into executable actions using the Factory and Command pattern, ensuring thread-safe communication and robust disconnection handling.

5. Test Coverage

Unit testing was heavily focused on the core logic components. We prioritized testing the Model and the Controller to guarantee strict adherence to the board game's rules and state consistency. As is standard practice and suggested by professors, Networking and View components show intentionally lower coverage, as GUI elements and asynchronous streams are generally tested through manual system tests rather than automated unit tests. The detailed coverage report is as follows:



The screenshot shows a code coverage report for the project 'java in PSP11'. The report is displayed in a dark-themed IDE. At the top, there's a tab labeled 'Coverage' and a sub-tab 'java in PSP11'. Below the tabs, there are icons for various coverage metrics. The main table has five columns: 'Element', 'Class, %', 'Method, %', 'Line, %', and 'Branch, %'. The table lists several packages and classes with their respective coverage percentages and counts. The 'it.polimi.ingsw' package is expanded, showing sub-packages like Controller, Model, Utils, Enums, CustomException, Networking, View, and Apps. The 'Controller' package shows 100% class coverage, 47% method coverage, 49% line coverage, and 38% branch coverage. The 'Model' package shows 94% class coverage, 80% method coverage, 81% line coverage, and 65% branch coverage. The 'Utils' package shows 83% class coverage, 60% method coverage, 72% line coverage, and 54% branch coverage. The 'Enums' package shows 72% class coverage, 63% method coverage, 53% line coverage, and 100% branch coverage. The 'CustomException' package shows 30% class coverage, 25% method coverage, 25% line coverage, and 100% branch coverage. The 'Networking' package shows 6% class coverage, 5% method coverage, 9% line coverage, and 19% branch coverage. The 'View' package shows 2% class coverage, 0% method coverage, 0% line coverage, and 0% branch coverage. The 'Apps' package shows 0% class coverage, 0% method coverage, 0% line coverage, and 0% branch coverage.

Element	Class, %	Method, %	Line, %	Branch, %
it.polimi.ingsw	43% (79/183)	33% (385/1150)	28% (1388/4930)	26% (397/1508)
Controller	100% (5/5)	47% (76/159)	49% (340/688)	38% (143/376)
Model	94% (50/53)	80% (254/314)	81% (805/986)	65% (202/307)
Utils	83% (5/6)	60% (12/20)	72% (91/126)	54% (40/74)
Enums	72% (8/11)	63% (21/33)	53% (78/145)	100% (0/0)
CustomException	30% (7/23)	25% (7/28)	25% (7/28)	100% (0/0)
Networking	6% (3/46)	5% (14/252)	9% (66/733)	19% (12/62)
View	2% (1/37)	0% (1/342)	0% (1/2129)	0% (0/641)
Apps	0% (0/2)	0% (0/2)	0% (0/95)	0% (0/48)